UNIVERSIDAD NACIONAL DE CÓRDOBA

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

Sistemas de Computación

TRABAJO PRÁCTICO N° 1:

RENDIMIENTO

Alumnos:

García, Lautaro Misael

Gomez, Dolores

Renaudo Gaggioli, Valentino

2026

Introducción

En este trabajo se analizará el rendimiento de sistemas computacionales a partir del estudio del tiempo de ejecución de programas y el uso de herramientas de profiling. El rendimiento de un sistema se define como su *capacidad para realizar una tarea en un determinado tiempo*, siendo este inversamente proporcional al mismo: a menor tiempo de ejecución, mayor rendimiento.

Se realizarán mediciones de tiempo de ejecución mediante herramientas de profiling con el objetivo de analizar cómo se distribuye el tiempo dentro de un programa e identificar las funciones más costosas.

Además, se estudiarán distintos benchmarks y su relación con tareas cotidianas, junto con la comparación del rendimiento de diferentes procesadores en la compilación del kernel de Linux. Finalmente, se analizará la aceleración obtenida entre arquitecturas utilizando la *Ley de Amdahl*, vinculando los resultados experimentales con los conceptos teóricos vistos en clase.

Marco teórico

Rendimiento

El rendimiento de un procesador depende de varios parámetros fundamentales:

- **Frecuencia de la CPU (fCPU):** Es la cantidad de ciclos de reloj que el procesador puede ejecutar por segundo. Se mide en Hertz (Hz). Una mayor frecuencia implica que el procesador puede realizar más operaciones por unidad de tiempo.
- **Periodo de la CPU (TCPU):** Es el tiempo que dura un ciclo de reloj y corresponde a la inversa de la frecuencia.

$$T_{CPU} = \frac{1}{f_{CPU}}$$

- **Ciclos por instrucción(CPI):** Las instrucciones de un programa se descomponen en operaciones más simples llamadas microinstrucciones. El CPI representa la cantidad promedio de ciclos de reloj necesarios para ejecutar una instrucción. Un menor CPI implica un mejor rendimiento.

$$CPI = \frac{\sum_{i=1}^{n} N° \, Instruccionesi * CPI_i}{N° \, Instruccion \, Total}$$

- **Número de instrucciones del programa** : Cuantas más instrucciones haya en el programa más tiempo se tarda en ejecutar, luego baja el rendimiento. El que tengamos un número reducido de instrucciones dependerá del programador y de que dispongamos de un buen compilador.

- **Multitarea**: Hace referencia a la capacidad que tiene un computador de atender simultáneamente varias tareas.

$$T_{instrucción} = CPI * T_{CPU}$$

$$T_{prog} = N° \, instrucciones * CPI * T_{CPU}$$

$$\eta_{prog} = \frac{1}{T_{prog}} = \frac{1}{N° \, instrucciones * CPI * T_{CPU}} = \frac{f_{CPU}}{N° \, instrucciones * CPI} s^{-1}$$

Throughput vs latencia

- **Latencia:** Es el tiempo que tarda el sistema en completar una única tarea desde que comienza hasta que finaliza. En el contexto de este trabajo, la latencia corresponde al tiempo que tarda el programa en ejecutarse completamente. Un sistema con menor latencia es más rápido para resolver una tarea individual.

- **Throughput (rendimiento o caudal):** Es la cantidad de tareas que el sistema puede completar por unidad de tiempo. Se expresa, por ejemplo, en “operaciones por segundo” o “programas ejecutados por segundo”. Un mayor throughput indica que el sistema puede procesar más trabajo en el mismo intervalo de tiempo.

En sistemas más complejos o multitarea, puede existir un compromiso entre ambas métricas. Por ejemplo, un sistema puede aumentar su throughput ejecutando múltiples tareas en paralelo, aunque esto implique un aumento en la latencia de cada tarea individual debido a la compartición de recursos.

Speedup y eficiencia

Speedup: Mide cuánto más rápido es un sistema mejorado respecto a uno original. Se define como la relación entre el tiempo de ejecución inicial y el tiempo de ejecución mejorado.

$$Speedup = \frac{T_{original}}{T_{mejorado}}$$

Eficiencia: Mide la utilización de un recurso.

$$Eficiencia = \frac{Speedup}{N}$$

donde N es el factor de mejora (por ejemplo, número de núcleos o incremento ideal esperado).

Ley de Amdahl

La **Ley de Amdahl** establece que el rendimiento de un sistema está limitado por la parte del programa que no puede paralelizarse.

En un programa, el tiempo de ejecución se puede dividir en dos partes:

- **Parte secuencial:**
  No se puede dividir ni ejecutar en paralelo. Siempre se ejecuta paso a paso.
- **Parte paralelizable:**
  Puede dividirse y ejecutarse simultáneamente en varios núcleos o con mejoras de hardware.

Consigna

Conseguir un esp32 o cualquier procesador al que se le pueda cambiar la frecuencia. Ejecutar un código que demore alrededor de 10 segundos. Puede ser un bucle for con sumas de enteros por un lado y otro con suma de floats por otro lado.

¿Qué sucede con el tiempo del programa al duplicar (variar) la frecuencia ?

Se realizó el experimento utilizando un ESP32, ejecutando un programa que realiza múltiples sumas con variables enteras y de punto flotante.

Frecuencia [MHz]	Enteros [s]	Floats [s]
80	13.91	16.43
160	6.91	8.17

Al aumentar la frecuencia de reloj del procesador, el tiempo de ejecución de un programa tiende a disminuir aproximadamente de forma inversamente proporcional, ya que el microcontrolador puede ejecutar un mayor número de instrucciones por segundo.

Sin embargo, esta relación no es perfectamente lineal, debido a factores como latencias de memoria, accesos a periféricos y sobrecarga del sistema.

Las operaciones con números enteros suelen escalar de manera más directa con la frecuencia, ya que son operaciones más simples y requieren menos ciclos de CPU.

En cambio, las operaciones con números de punto flotante dependen en gran medida de la unidad de punto flotante (FPU). Aunque el ESP32 posee soporte para operaciones en coma flotante, estas suelen ser más costosas en términos de ciclos de ejecución que las operaciones enteras.

Adicionalmente, se realizó la misma prueba en una placa basada en el microcontrolador RP2350 con núcleos ARM Cortex-M33, obteniéndose los siguientes resultados:

Frecuencia [MHz]	Enteros [s]	Floats [s]
80	13.750	13.750
160	6.875	6.875

En este caso, los tiempos de ejecución para operaciones con enteros y punto flotante resultaron prácticamente iguales, lo que sugiere una implementación más eficiente o mejor integrada de la unidad de punto flotante en esta arquitectura. Además, se mantiene el comportamiento inversamente proporcional entre frecuencia y tiempo de ejecución, similar al observado en el ESP32, aunque con menor penalización relativa en operaciones de punto flotante.

Profiling

El profiling permite medir y analizar el comportamiento real de un programa durante su ejecución, proporcionando información relevante como:

- Tiempo total de ejecución
- Tiempo de CPU consumido por cada función
- Frecuencia de llamadas a funciones
- Identificación de cuellos de botella

Existen dos técnicas principales para el análisis del tiempo de ejecución a nivel de funciones: **inyección de código (instrumentación)** y **muestreo (sampling)**.

Tipos:

- Inyección de código → gprof: Estos perfiladores requieren volver a compilar el programa con banderas especiales. En cada punto donde se realiza una llamada/retorno de función, se inserta este código para verificar la hora actual del sistema (hasta microsegundos) y registrar esa información. El programa de creación de perfiles lee y resume el archivo de registro, generando un informe sobre cuánto tiempo se dedicó a cada función/método, cuántas veces se llamó, etc.

- Sampling → perf: Estos requieren privilegios de nivel de sistema operativo. El programa se ejecuta en un modo especial, con interrupciones del sistema operativo generadas a intervalos fijos. En cada una de estas interrupciones, verifica y registra qué función/método está ejecutando el programa actualmente. Esto acumula datos estadísticos sobre cuánto tiempo se dedicó a cada función. El perfilador analiza e informa sobre los datos estadísticos.

A partir del perfil generado por gprof, se puede analizar el comportamiento temporal del programa, identificando qué funciones consumen mayor tiempo de CPU y cómo se distribuye la ejecución. El programa analizado está compuesto por dos archivos fuente. En el archivo principal se definen las funciones `func1`, `func2` y `main`, mientras que en un segundo archivo se implementa `new_func1`.

Call graph: muestra cómo se distribuye el tiempo entre funciones y sus llamadas.

```
main
├── func1
│   └── new_func1
└── func2
```

Esto coincide exactamente con el código fuente. Además, el grafo permite distinguir entre:

- **Tiempo propio de una función (*self*)**, es decir, el tiempo que pasa ejecutando su propio código;

- **Tiempo de hijos (*children*)**, es decir, el tiempo consumido por las funciones llamadas desde ella.

En el caso de `func1`, `gprof` informa aproximadamente:

- 1.69 s de tiempo propio
- 0.12 s en funciones hijas, correspondientes a `new_func1`

Esto significa que casi todo el tiempo de `func1` proviene de su gran bucle interno, y solo una pequeña parte corresponde a la función que llama al final. Por lo tanto, si se quisiera optimizar `func1`, el mayor beneficio vendría de actuar sobre su propio bucle y no sobre `new_func1`.

En `func2`, todo el tiempo es propio, ya que no llama a ninguna otra función. Por ello, el costo medido corresponde exclusivamente a su bucle.

Análisis de Flat Profile (Gomez)

```
dolores@DESKTOP-SNSD2LV:

Flat profile:

Each sample counts as 0.01 seconds.
  %   cumulative   self              self     total
 time   seconds   seconds    calls   s/call   s/call  name
 54.26      1.72     1.72        1     1.72     1.82  func1
 39.43      2.97     1.25        1     1.25     1.25  func2
  3.15      3.07     0.10        1     0.10     0.10  new_func1
  3.15      3.17     0.10                             main

 %         the percentage of the total running time of the
time       program used by this function.

cumulative a running sum of the number of seconds accounted
 seconds   for by this function and those listed above it.

 self      the number of seconds accounted for by this
seconds    function alone.  This is the major sort for this
           listing.

calls      the number of times this function was invoked, if
           this function is profiled, else blank.

 self      the average number of milliseconds spent in this
ms/call    function per call, if this function is profiled,
           else blank.

 total     the average number of milliseconds spent in this
ms/call    function and its descendents per call, if this
           function is profiled, else blank.

name       the name of the function.  This is the minor sort
           for this listing. The index shows the location of
           the function in the gprof listing. If the index is
           in parenthesis it shows where it would appear in
           the gprof listing if it were to be printed.
^L
Copyright (C) 2012-2024 Free Software Foundation, Inc.

Copying and distribution of this file, with or without modification,
are permitted in any medium without royalty provided the copyright
:
```

Figura 1: Resultados Flat Profile

```
Copying and distribution of this file, with or without modification,
are permitted in any medium without royalty provided the copyright
notice and this notice are preserved.
^L
                     Call graph (explanation follows)


granularity: each sample hit covers 4 byte(s) for 0.32% of 3.17 seconds

index % time    self  children    called     name
                                                 <spontaneous>
[1]    100.0    0.10    3.07                 main [1]
                1.72    0.10       1/1           func1 [2]
                1.25    0.00       1/1           func2 [3]
-----------------------------------------------
                1.72    0.10       1/1           main [1]
[2]     57.4    1.72    0.10       1         func1 [2]
                0.10    0.00       1/1           new_func1 [4]
-----------------------------------------------
                1.25    0.00       1/1           main [1]
[3]     39.4    1.25    0.00       1         func2 [3]
-----------------------------------------------
                0.10    0.00       1/1           func1 [2]
[4]      3.2    0.10    0.00       1         new_func1 [4]
-----------------------------------------------

 This table describes the call tree of the program, and was sorted by
 the total amount of time spent in each function and its children.
```

Figura 2: Flat Profile

Funcion	Tiempo(%)	Tiempo(s)	Llamadas
func1	54.26%	1.72 s	1
func2	39.43%	1.25 s	1
new_func1	3.15%	0.10 s	1
main	3.15%	0.10 s	-

`func1` es la función más costosa, consumiendo más de la mitad del tiempo total (53.65%).

`func2` también representa un costo importante (39.37%).

El resto de las funciones tienen un impacto despreciable (<5%).

Cada función se ejecuta una sola vez, lo que indica que el tiempo depende del trabajo interno de cada función, no de la cantidad de llamadas.

La información obtenida del *flat profile* y del *call graph* confirma que el rendimiento del programa está dominado por los bucles de gran tamaño definidos en `func1` y `func2`. El estudio permite concluir que, si se quisiera optimizar el programa, las mejoras deberían enfocarse en `func1` y `func2`, ya que cualquier modificación en `main` o `new_func1` tendría un impacto poco relevante sobre el tiempo total de ejecución.

Benchmark

Tarea diaria	Benchmark representativo	Clasificación
LTspice	Benchmark kernel o de núcleo	Se evalúa una parte específica: resolución de ecuaciones del circuito El núcleo del cálculo es numérico (método nodal, matrices) No incluye tareas externas complejas (UI, archivos grandes, etc.)
KLayout	Benchmark reducido	Se trabaja sobre un subconjunto del flujo completo de diseño Se evalúan reglas específicas (DRC, verificación geométrica) No es el flujo completo de fabricación
Visual Studio Code	Benchmark reducido	Se evalúa una parte del sistema: carga de archivos e indexado del proyecto. No representa el funcionamiento completo del programa. Involucra CPU y acceso a disco.
GNU Octave	Benchmark kernel o de núcleo	Se evalúa el núcleo de cálculo matemático mediante operaciones numéricas. El rendimiento depende principalmente del uso de CPU/FPU. No incluye tareas externas complejas.
Transferencia de archivos (copias a disco externo)	fio / CrystalDiskMark	Benchmark de sistema
Descargas (qBittorrent, tráfico de red)	iperf3	Benchmark de sistema

Clasificación:

-BENCHMARK SINTÉTICOS: generalmente pequeños, escritos originalmente utilizados para comprobar el comportamiento ante un tipo determinado de carga.

-BENCHMARKS REDUCIDOS

-BENCHMARK KERNEL O DE NÚCLEO

-PROGRAMAS REALES

Comparación de CPUs

Para comparar el rendimiento de distintos procesadores se pueden utilizar plataformas como OpenBenchmarking.org que recopilan resultados reales de benchmarks ejecutados en diferentes sistemas.

En este caso se analizan los siguientes procesadores:

- **Intel Core i5-13600K**
- **AMD Ryzen 9 5900X**
- **AMD Ryzen 9 7950X**

El **percentil de rendimiento** indica qué tan bien se posiciona un procesador respecto a otros.

La **arquitectura** (por ejemplo, Zen 3, Zen 4, Raptor Lake) define el diseño interno del procesador.

Incluye aspectos como:

- Organización de los núcleos
- Manejo de memoria
- Pipeline de ejecución
- Optimizaciones internas

Una arquitectura más moderna suele ofrecer mejor rendimiento incluso a igual frecuencia, ya que puede ejecutar más instrucciones por ciclo (mayor IPC).

Los **núcleos** son unidades de procesamiento independientes dentro del CPU. Mientras más núcleos haya, hay mejor rendimiento en tareas paralelas.

En CPUs híbridas (como Intel):

P-cores (Performance) → alto rendimiento

E-cores (Efficient) → menor consumo, tareas secundarias

Los **hilos** representan la cantidad de tareas que el procesador puede manejar simultáneamente. Mientras más hilos, mejor multitarea y mejor rendimiento encargas paralelas.

La **frecuencia base** es la velocidad mínima a la que opera el procesador (en GHz).

La **frecuencia boost** es la frecuencia máxima que puede alcanzar el procesador en condiciones ideales.

En el **tipo de diseño** se describe cómo están organizados los núcleos.

La **tecnología de fabricación** se refiere al tamaño del proceso de fabricación. A menor tamaño, hay mayor eficiencia.

Característica	Intel Core i5-13600K	AMD Ryzen 9 5900X	AMD Ryzen 9 7950X
Arquitectura	Raptor Lake	Zen 3	Zen 4
Núcleos	14 (6P + 8E)	12	16
Hilos	20	24	32
Frecuencia base	~3.5 GHz	~3.7 GHz	~4.5 GHz
Frecuencia boost	~5.1 GHz	~4.8 GHz	~5.7 GHz
Tipo de diseño	Híbrido (P + E cores)	Tradicional	Tradicional
Tecnología	Intel 7 (~10 nm)	7 nm	5 nm
Percentil (OpenBenchmarking)	~58%	~56%	~74%

De acuerdo con datos de OpenBenchmarking.org, el AMD Ryzen 9 7950X presenta un percentil de rendimiento significativamente superior (74th) en comparación con el Intel Core i5-13600K (58th) y el Ryzen 9 5900X (56th).

Esta diferencia refleja un salto importante en rendimiento debido a su mayor cantidad de núcleos e hilos, así como a su arquitectura más moderna (Zen 4). Esto implica un mayor rendimiento por ciclo de reloj (IPC), permitiendo ejecutar más instrucciones en el mismo tiempo.

En tareas como la compilación del kernel de Linux, que son altamente paralelizables, esto se traduce en tiempos de ejecución considerablemente menores, posicionando al Ryzen 9 7950X como la mejor opción entre los tres procesadores analizados.

Además, la tecnología de fabricación influye en la eficiencia energética y la densidad de transistores. El proceso de 5 nm del 7950X permite mayor rendimiento con menor consumo en comparación con los 7 nm y ~10 nm de los otros modelos.

Conclusión

El análisis del rendimiento de un sistema no puede basarse únicamente en la teoría. Si bien la complejidad algorítmica da una idea de cómo se comporta un programa, en la práctica influyen muchos otros factores como el hardware, la memoria y el sistema operativo.

En este sentido, el **profiling** resulta una herramienta clave, ya que permite ver qué partes del código consumen más tiempo durante la ejecución. Esto facilita detectar cuellos de botella y enfocar las optimizaciones donde realmente vale la pena.

Por otro lado, los **benchmarks** permiten comparar el rendimiento entre distintos sistemas o procesadores. Dependiendo del tipo de benchmark, se puede evaluar desde el rendimiento puro del CPU hasta el comportamiento en tareas reales.

Además, al comparar CPUs se observa que el rendimiento depende de varios factores combinados, como la arquitectura, la cantidad de núcleos, la frecuencia y la capacidad de trabajar en paralelo. En general, procesadores más modernos y con más núcleos tienden a rendir mejor en tareas exigentes.